Introdução à Programação Orientada a Objetos em Java
Parte 2

I. Encapsulamento
II. Herança
III. Classes e Métodos Abstratos
IV. Referências

I. Encapsulamento

Encapsulamento ou "*data hiding*" é um conceito bastante importante em orientação a objetos. É utilizado para restringir o acesso às declarações de uma classe e à própria classe.

Em Java, a visibilidade padrão de classes, atributos e métodos está restrita a todos os membros que fazem parte de um mesmo pacote.

A palavra-chave *public* modifica essa visibilidade de forma a ampliá-la, deixando-a sem restrições.

Uma classe definida como pública pode ser utilizada por qualquer objeto de qualquer pacote.

Em Java, uma unidade de compilação (um arquivo fonte com extensão .java) pode ter no máximo uma classe pública, cujo nome deve ser o mesmo do arquivo (sem a extensão).

As demais classes na unidade de compilação, não públicas, são consideradas classes de suporte para a classe pública e têm a visibilidade padrão.

Um atributo público de uma classe pode ser diretamente acessado e manipulado por objetos de outras classes. Assim como, um método público de uma classe pode ser aplicado a um objeto dessa classe a partir de qualquer outro objeto de outra classe. O conjunto de métodos públicos de uma classe determina o que pode ser feito com objetos da classe, ou seja, determina o seu comportamento.

A palavra-chave *protected* restringe a visibilidade do membro modificado, método ou atributo, exclusivamente a objetos da própria classe que contém sua definição e às suas subclasses.

A palavra-chave *private* restringe a visibilidade do membro modificado, método ou atributo, exclusivamente a objetos da própria classe que contém sua definição.

Modificadores de visibilidade

Os modificadores de visibilidade desempenham papel importante na POO ao controlar o acesso aos membros (propriedades e métodos) de uma classe. Eles ajudam a estabelecer o encapsulamento e a modularidade do código, fornecendo restrições sobre como os membros podem ser acessados e manipulados.

Os modificadores de visibilidade também são chamados de modificadores de acessibilidade, pois na prática eles restringem o acesso aos membros da classe.

- ***public***: A classe, método ou atributo, são sempre acessíveis em todos os métodos de todas as classes;
- ***private***: A classe, método ou atributo, são acessíveis somente por métodos da própria classe. É o nível mais rígido de encapsulamento;
- ***protected***: A classe, método ou atributo, são acessíveis nos métodos da própria classe e suas subclasses (herança);
- ***pakage***: A classe, método ou atributo, são acessíveis somente por métodos das classes que pertencem ao mesmo "*pakage*".

Package (Pacote)

No desenvolvimento de pequenas atividades ou aplicações, é viável manter o código e suas classes no diretório corrente. No entanto, para grandes aplicações é preciso organizar as classes de maneira a:

1. evitar problemas com nomes duplicados de classes, e
2. localizar o código da classe de forma eficiente.

Em Java, a solução para esse problema está na organização de classes e interfaces em pacotes. Essencialmente, uma classe Tal que pertence a um pacote nome.do.pacote tem o nome completo formado por nome.do.pacote.Tal e o compilador Java espera encontrar o arquivo Tal.class em um subdiretório nome/do/pacote. Este, por sua vez, deve estar localizado sob um dos diretórios especificados na variável de ambiente CLASSPATH.

Como estamos usando o NetBeans, toda essa organização é feita automaticamente pela IDE.

Getters e Setters

Na linguagem Java, há uma convenção de nomenclatura para os métodos que têm como finalidade acessar ou alterar as propriedades de um objeto.

Segundo essa convenção, os nomes dos métodos que permitem a consulta das propriedades de um objeto devem possuir o prefixo ***get***. Analogamente, os nomes dos métodos que permitem a alteração das propriedades de um objeto devem possuir o prefixo ***set***.

Na maioria dos casos, é muito conveniente seguir essa convenção, pois os desenvolvedores Java já estão acostumados com essas regras de nomenclatura e o funcionamento de muitas bibliotecas do Java depende fortemente desse padrão.

Vejamos abaixo como ficariam os **get** e **set** da classe Pessoa depois de encapsulados:

```
public class Pessoa{
    private String nome;
    private int idade;
    Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
    public void imprimir() {
        System.out.println(this.nome + " " + this.idade);
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public int getIdade() {
        return idade;
    }
    public void setIdade(int idade) {
        this.idade = idade;
    }
}
```

Assim ao instanciarmos um objeto da classe Pessoa, precisaremos utilizar os métodos **get** e **set** para termos acesso aos seus atributos:

```
public class Testes {

    public static void main(String[] args) {
        Pessoa p = new Pessoa();

        p.setNome("Ana");
        p.setIdade(18);

        p.imprimir();

        System.out.println(p.getNome());
    }
}
```

II. Herança

A herança é um recurso da POO (Programação Orientada a Objetos) que permite que uma classe herde propriedades e métodos de outra classe. Isso possibilita a criação de uma hierarquia de classes, onde as subclasses herdam o comportamento e as características da superclasse.

Na programação Java, a herança é codificada usando a palavra reservada *extends* (estender) seguida pelo nome da classe a ser herdada. No exemplo a seguir a instrução *public class* Cliente *extends* Pessoa é usada para definir que a classe Cliente herda/estende da classe Pessoa.

```
public class Pessoa {

    protected String nome;
    protected int idade;

    Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}

public class Cliente extends Pessoa {

    private float saldo;

    Cliente(String nome, int idade, float saldo) {
        super(nome, idade);
        this.saldo = saldo;
    }

    public void print() {
       System.out.println(this.nome + " " +
      this.idade + " " + this.saldo);
    }
}
```

Considerações sobre o *super*:

- A palavra reservada super é usada na subclasse para referenciar o seu objeto da superclasse. Porém, super.nome resultará como não existente, pois as propriedades da superclasse são consideradas propriedades do objeto this, logo não estão disponíveis pelo objeto super.

No exemplo a seguir adicionou-se o método imprimir na superclasse. Para chamar esse método na subclasse é necessário preceder o nome do método pela palavra reservada *super* ou *this*.

```
public class Pessoa {

    protected String nome;
    protected int idade;

    Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public void imprimir() {
        System.out.println(this.nome + " " + this.idade);
    }
}

public class Cliente extends Pessoa {

    private float saldo;

    Cliente(String nome, int idade, float saldo) {
        super(nome, idade);
        this.saldo = saldo;
    }

    public void print() {
        this.imprimir();
        super.imprimir();
        System.out.println(this.nome + " " + this.idade + " " +
        this.saldo);
    }
}
```

Observações:

• A superclasse não tem a capacidade de saber quais são as classes filhas;
• A linguagem Java não suporta herança múltipla, isto é, uma subclasse não pode herdar de mais de uma superclasse;
• As classes podem herdar hierarquicamente. No exemplo a seguir os objetos da classe Cliente possuem os métodos das classes Pessoa e Documento. Isso acontece pelo fato dos métodos da classe Documento serem herdados pela classe Pessoa e estes, por sua vez, são herdados pela classe Cliente:

```
public class Documento {
    String tipo;
    String numero;
    String orgaoemissor;
    String estado;

    public Documento(String tipo, String numero, String orgaoemissor,
String estado) {
        this.tipo = tipo;
        this.numero = numero;
        this.orgaoemissor = orgaoemissor;
        this.estado = estado;
    }
}

public class Pessoa extends Documento{

    protected String nome;
    protected int idade;

    Pessoa(String nome, int idade, String tipo, String numero, String
orgaoemissor, String estado) {
        super(tipo, numero, orgaoemissor, estado);
        this.nome = nome;
        this.idade = idade;
    }

    public void imprimir() {
        System.out.println(this.nome + " " + this.idade);
    }
}

public class Cliente extends Pessoa {

    private float saldo;

    Cliente(String nome, int idade, float saldo, String tipo, String
numero, String orgaoemissor, String estado) {
        super(nome, idade, tipo, numero, orgaoemissor, estado);
        this.saldo = saldo;
    }

    public void print() {
        this.imprimir();
        super.imprimir();
        System.out.println(this.nome + " " + this.idade + " " +
this.saldo);
    }
}
```

O principal papel da herança na POO é possibilitar a reutilização de código. O programador pode utilizar comportamentos já codificados ao programar novas funcionalidades. Imagine a necessidade de

codificar uma classe Aluno, poderíamos estender da classe Pessoa e evitar ter de programar as propriedades e métodos existentes na classe Pessoa.

III. Classes e Métodos Abstratos

Classe Abstrata

Uma classe abstrata é definida usando a palavra reservada *abstract* antes da declaração da classe. Ela pode ter alguns métodos sem implementação (sem corpo – o corpo é definido pelo par de chaves {}).

```
public abstract class Pessoa {

    protected String nome;
    protected int idade;

    Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // um método abstrato não possui corpo
    public abstract void print();
}

public class Testes {

    public static void main(String[] args) {
        // errado: uma classe abstrata não pode ser instanciada usando new
        Pessoa p = new Pessoa("Ana", 18);

        p.print(); // errado: o método print não possui corpo
    }
}
```

Uma classe abstrata não pode ser instanciada diretamente pelo fato dela poder ter métodos sem corpo. A classe abstrata é projetada para ser estendida e fornecer uma estrutura comum para suas subclasses.

No exemplo a seguir a classe **Cliente** estende (herda) a classe Pessoa, logo ela precisará implementar o método print, pois as subclasses são obrigadas a implementar todos os métodos sem implementação da superclasse abstrata.

```
public class Cliente extends Pessoa {

    private float saldo;

    Cliente(String nome, int idade, float saldo) {
        super(nome, idade);
        this.saldo = saldo;
    }

    @Override
    public void print() {
```

```
        System.out.println(this.nome + " " + this.idade + " " +
this.saldo);
    }
}

public class Testes {

    public static void main(String[] args) {
        // errado: uma classe abstrata não pode ser instanciada usando new
        Pessoa p = new Pessoa("Ana", 18);

        p.print(); // errado: o método print não possui corpo

        // certo: a classe Cliente é concreta
        Cliente c = new Cliente("Ana", 18, 950);

        c.print(); // certo: o método print possui implementação na classe
Cliente
    }
}
```

As classes e métodos abstratos fornecem uma maneira de definir uma estrutura comum e comportamentos padrões para suas subclasses, promovendo a reutilização de código e fornecendo uma base sólida para a hierarquia de classes.

IV. Referências

Partes desta apostila, foram baseadas no material didático do prof. Dr. Arley Ferreira de Souza coordenador do curso de DSM da Fatec de Jacareí.

Java: How to program, 4th. Ed., Harvey M. Deitel e Paul J. Deitel, Prentice-Hall, 2001, ISBN 0-13-034151-7.

Design patterns: Elements of reusable object-oriented software, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Addison-Wesley, 1995, ISBN 0-201-63361-2.

Object-oriented design in Java, Stephen Gilbert, Bill McCarty, The Waite Group Press, 1998, ISBN 1-57169-134-0.

The Unified Modeling Language User Guide, Grady Booch, James Rumbaugh, Ivar Jacobson, Addison-Wesley, 1999, ISBN 0-201-57168-4.

The Java Tutorial: A practical guide for programmers, Mary Campione e Kathy Walrath, acesso em 30/04/2024.

Java (linguagem de programação), Wikipédia, acesso em 30/04/2024.